



Operating Systems Lab

Course Code: CSE-412

Experiment Title: Round Robin (RR) CPU Scheduling Algorithm

Student Name: Md. Shoyaif Rahman

ID: 0432310005101050

Instructor Name: Afroja Ahmed Smrity

University Name: University of Information Technology and Sciences (UITS)

Experiment No: 04

Experiment Title: Implementation of Round Robin Scheduling (with and without Arrival Time)

Date of Performance: 21/04/26

Date of Submission: 25/04/26

Objective of the Experiment

- To understand the Round Robin (RR) scheduling algorithm and the concept of time quantum.
- To implement RR for processes with arrival time 0.
- To modify the program to handle processes with different arrival times.
- To analyze the impact of small vs. large time quanta.

Theory / Background

Round Robin (RR) is a preemptive scheduling algorithm designed for time-sharing systems. Each process is given a small unit of CPU time called a time quantum (TQ). Processes are kept in a ready queue. The CPU cycles through the queue, giving each process a maximum of TQ time. If a process does not finish in its quantum, it is preempted and moved to the back of the queue. This ensures fairness and fast response time. A very small TQ causes high context switching overhead, while a very large TQ makes RR behave like FCFS.

Procedure**1.Part 1 (AT=0):**

- Input number of processes, BT, and TQ. Assume AT=0.
- Use a do-while loop to cycle through processes.
- If $\text{rem_bt}[i] > \text{TQ}$, decrement it by TQ and increase time. Else, add remaining time to WT.
- Calculate TAT and display results.

2.Part 2 (Different AT):

- Input AT, BT, and TQ.
- Use a while($\text{remain} > 0$) loop. For each process i , if $\text{at}[i] \leq \text{time}$ and $\text{rt}[i] > 0$, execute for $\min(\text{rt}[i], \text{tq})$.
- Update time, remaining time, and print completion messages.

Program Code

```
#include <stdio.h>
int main() {
    int n = 4, tq = 2;
    int bt[4] = {5,4,2,1}, rem_bt[4], wt[4] = {0}, tat[4] = {0};
    int t = 0, done;
    for(int i = 0; i < n; i++) rem_bt[i] = bt[i];
    do {
        done = 1;
        for(int i = 0; i < n; i++) {
```

```

    if(rem_bt[i] > 0) {
        done = 0;
        if(rem_bt[i] > tq) {
            t += tq;
            rem_bt[i] -= tq;
        } else {
            t += rem_bt[i];
            wt[i] = t - bt[i];
            rem_bt[i] = 0;
        }
    }
}
} while(!done);
for(int i = 0; i < n; i++) tat[i] = wt[i] + bt[i];
printf("P\tBT\tWT\tTAT\n");
for(int i = 0; i < n; i++) printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
return 0;
}

```

Input and Output:

Output (AT=0, TQ=2):

P	BT	WT	TAT
P1	5	7	12
P2	4	6	10
P3	2	4	6
P4	1	5	6

Analysis / Discussion

Round Robin ensures that no process is starved, as every process eventually gets CPU time. For the set of processes, RR with TQ=2 provided fairness but increased context switches compared to FCFS. The waiting time for P1 (7 ms) was higher than in FCFS due to preemption. The choice of time quantum is critical. A small quantum (e.g., 1ms) leads to excellent response time but high overhead (>90% context switching). A large quantum (e.g., 100ms) reduces overhead but increases response time, making the system unresponsive. An optimal quantum (e.g., 20ms in a lab scenario) balances throughput, response time, and context switching overhead.

Conclusion

The Round Robin algorithm was successfully implemented for both uniform and varying arrival times. RR is excellent for interactive systems due to its fairness and predictable response time, but its performance is highly dependent on the optimal selection of the time quantum, which must balance overhead and responsiveness.